ECE Written Comprehensive Exam
Research Proposal

Advancements for Image Classification in Real-World Settings: Innovations in Distributed Dataset Acquisition and Cost-Sensitive Machine Learning

Zi K. Deng

Department of Electrical & Computer Engineering

The University of Arizona

Tucson, Arizona

Faculty Advisor: Jeffrey J. Rodriguez

Associate Professor, Department of Electrical & Computer Engineering

Comprehensive Exam Committee:

Jerzy W. Rozenblit, Professor of Electrical & Computer Engineering

Abhijit Mahalanobis, Associate Professor of Electrical & Computer Engineering

Chicheng Zhang, Assistant Professor of Computer Science

April 16, 2026

Abstract

Deep learning models for image classification have achieved remarkable performance on curated benchmarks, yet deploying these models in applied domains such as biodiversity monitoring and medical screening exposes two significant gaps. First, constructing training datasets from distributed web sources often requires downloading images from multiple heterogeneous servers. Many of these servers enforce rate limits that existing tools neither detect nor respect, which can lead to systematic download failures and biased training data. Second, standard classification objectives treat all misclassification errors as equally costed. However, specific errors often carry consequences that differ by orders of magnitude, such as misidentifying a dangerous species as harmless or undergrading a severe disease stage. This proposal presents two contributions addressing these gaps.

In our first contribution, we develop FLOW-DC (Flexible Large-scale Orchestrated Workflow for Data Collection), a distributed image downloading framework for constructing ML-ready datasets from URL manifests hosted across heterogeneous servers. Existing tools such as img2dataset lack adaptive rate-control mechanisms: they treat server-side throttling as permanent failure, leading to download failures concentrated on the small institutional servers that lack large-scale infrastructure. To address this, FLOW-DC introduces PAARC (Policy-Aware Adaptive Request Controller), a BBR (Bottleneck Bandwidth and Round-trip propagation time) inspired application-layer congestion control algorithm that dynamically adjusts per-host request concurrency based on TTFB (time to first byte) latency, HTTP status codes, and connection-level error signals. PAARC operates within the TaskVine distributed workflow manager, enabling elastic execution across heterogeneous compute environments (university HPC clusters, cloud instances, and local workstations) without requiring a shared filesystem or synchronized start times. FLOW-DC has been deployed at production scale for the construction of the 38.7-million-image Biotrove dataset and will be evaluated against img2dataset on both content delivery network-based and server-diverse workloads using server-side failure rate, scaling efficiency, and class distribution fidelity as the primary metrics.

In our second contribution, we address the inability of standard image classifiers to distinguish between misclassification errors of differing cost. In many applied domains, certain prediction errors are far more consequential than others. For example, misidentifying a deadly spider as harmless could delay life-saving treatment, while the reverse error results only in unnecessary caution. Standard cross-entropy training treats all errors as equally costly and provides no mechanism for a domain expert to encode their knowledge on misclassification costs. To address this, we present NICME (Non-Identical Cost Matrix Embedding), a cost-sensitive classification framework that accepts a user-defined misclassification cost matrix and integrates it into both the logit computation and a dedicated regularization term. NICME utilizes LoRA (Low-Rank Adaptation) for fine-tuning to prevent the model from memorizing the training data entirely. NICME will be evaluated across four datasets spanning binary classification to 20-class identification tasks in domains including spider species classification, diabetic retinopathy grading, and pharmacy medication dispensing. It will be compared against standard cross-entropy, as well as the regularization and logit adjustment components applied individually.

Keywords: cost-sensitive learning, congestion control, image classification, deep learning, cost matrix, distributed download

1. Research Topic and Its Importance

1.1 Introduction

Deep learning models for computer vision have improved at a rapid pace over the past two decades, progressing from the convolutional architectures that first achieved production-grade image classification such as AlexNet and ResNet [1, 2], to the dominance of transformer-inspired architectures such as ViT, Swin, and ConvNeXt [3–5], and now to the current ecosystem of compute-intensive self-supervised foundation models such as MAE, CLIP, and DINO [6–8]. These advances have been matched by a corresponding growth in dataset scale: from CIFAR-10's 60,000 images totaling approximately 170 MB [9] to LAION-5B's 5.85 billion image-text pairs totaling approximately 220 TB [10]. Each generation of models depends on increasingly large and more diverse training corpora to achieve better performance and stronger generalization.

However, the infrastructure and assumptions underlying these benchmark datasets do not reflect the conditions that researchers commonly encounter when building classification systems for applied domains. Benchmark datasets are distributed as self-contained archives with uniform formatting, generally balanced or well-characterized class distributions, and have their images available from high-capacity servers and fast networks [11]. In contrast, constructing a training dataset for domains such as biodiversity monitoring or practical medical screening often requires downloading millions of images from heterogeneous web sources, such as university servers, museum repositories, and citizen-science platforms [12]. Many of these resources operate on limited server infrastructure and enforce rate limits that existing download tools neither detect nor respect [13]. Once such a dataset is assembled, another gap often emerges at the model training stage: standard classification objectives implicitly assume that all misclassifications are equally damaging, an assumption that breaks down in applied settings where some errors carry severe real-world consequences. Misidentifying a dangerous species as harmless carries a significantly different implication than confusing two harmless species [14]; similarly, failing to detect a severe stage of diabetic retinopathy carries different consequences than overgrading a mild case [15]. The theoretical foundations for incorporating such asymmetric costs into classification have existed for decades [16], yet their integration into modern deep learning architectures remains limited.

This research addresses both gaps through two complementary lines of investigation. First, we have developed a distributed image downloading pipeline that utilizes an application-layer congestion control algorithm to adapt request rates to individual host server capacity and allowances, enabling researchers and groups operating outside of resource-rich organizations to construct large-scale training datasets from server-diverse web sources in a timely manner without triggering rate-limit violations. Second, we are developing a cost-sensitive learning (CSL) framework that accepts a user-defined $N \times N$ misclassification cost matrix as a training hyperparameter, incorporated into a training pipeline that utilizes both cost-sensitive regularization and logit adjustment. We believe the combination of a robust data downloader and a CSL framework decoupled from class imbalance will provide meaningful improvements to practical image classification and strengthen the end-to-end machine learning (ML) pipeline across a variety of application domains.

1.2 Limitations of Current Cost-Sensitive Learning and Dataset Acquisition Paradigms

Our research has identified two specific technical problems that arise when tackling image classification pipelines for real-world applications. The first is the lack of a dataset construction tool that can utilize adaptive per-host congestion control to mitigate server overload and throttling. To be an improved alternative to leading dataset downloaders, such a tool must be able to execute in a distributed manner, ideally across heterogeneous compute environments to accommodate varying infrastructure constraints. The problem with existing tools is

that they lack adaptive rate-control mechanisms, treating server-side throttling as permanent failures, and require homogeneous compute cluster infrastructure such as Apache Spark [17, 18] that is either unavailable or impractical to set up in many academic settings. The second is the absence of cost-sensitive deep learning methods that operate with user-defined cost matrices on modern vision architectures. Existing approaches either derive cost weights from class frequency statistics rather than domain knowledge [19, 20], or have only been validated on outdated network architectures [21]. Accordingly, our research objectives are the following:

Objective 1: Distributed Downloading with Adaptive Congestion Control. Our first objective was to develop a distributed image download system that achieves download throughput comparable to the current state-of-the-art distributed downloaders, but significantly reduces the failure rate from server-side faults such as HTTP 429, 5XX, and connection reset errors.

Objective 2: Cost-Sensitive Classification with User-Defined Cost Matrices. Our second objective is to develop a cost-sensitive image classification framework that aims to attain higher recall on user-designated high-cost classes than standard cross-entropy loss training and rivals existing cost-sensitive methods. To achieve this, we specifically introduce an $N \times N$ cost matrix as a hyperparameter to be utilized during model training.

2. Review of Current Technology and Motivation

2.1 Distributed Image Dataset Construction

The growth of web-scale vision datasets has created a need for tools that can convert URL manifests, tabular files listing the remote locations of millions of images, into locally stored, ML-ready training sets. Two systems have emerged as the primary tools for this task: img2dataset, which underpins the construction of most large-scale web-crawled vision datasets, and the Imageomics distributed-downloader, which was developed specifically for biodiversity image acquisition from institutional repositories.

The tool img2dataset, created by Beaumont and central to the construction of LAION-400M, LAION-5B, and DataComp, is the most widely adopted open-source tool for large-scale image downloading and packaging [10, 22]. Its architecture follows a staged pipeline: a reader ingests URL lists from one of thirteen supported input formats via PyArrow, a distributor dispatches shards to a pool of worker processes, and within each process a thread pool of up to 256 threads performs concurrent HTTP downloads using Python's synchronous `urllib.request` library [22]. Downloaded images are resized via OpenCV and written to one of five output formats, with WebDataset tar archives recommended for datasets exceeding one million samples. With optimized network speeds, img2dataset achieves approximately 90 samples per second per CPU core, scaling to roughly 1,300 images per second on a 16-core machine and approximately 10,000 images per second across a 10-node Spark cluster [10]. The tool's largest documented deployment downloaded LAION-5B's 5.85 billion image-text pairs using ten AWS c6i.4xlarge instances over approximately one week, producing 220 TB of output at 384-pixel resolution.

A notable architectural characteristic of img2dataset is its lack of any congestion control or host-aware request scheduling. All URLs within a shard are downloaded as fast as the thread pool allows, with no throttling based on server response times, HTTP status codes, or per-domain rate limits. When a server responds with HTTP 429 (Too Many Requests), img2dataset treats this as a standard failure, no different than if the content doesn't exist; by default, no retries are attempted (`retries=0`). This design is well suited to downloading from content delivery networks (CDNs) such as Cloudflare and AWS CloudFront, which

tolerate high concurrency without rate limiting. However, when the target data resides on smaller institutional servers, as is characteristic of biodiversity repositories, museum collections, and regional medical archives, the absence of rate control produces cascading failures: excessive rate-limiting errors can lead to permanently failed downloads, and in severe cases, IP-level blocks from the host server.

The Imageomics distributed-downloader, developed by the TreeOfLife project for constructing biodiversity datasets from GBIF (Global Biodiversity Information Facility) occurrence records, addresses a different deployment scenario [12]. This tool is designed around Slurm-based queue submission with MPI (message passing interface) for inter-node coordination, making it well suited to university HPC clusters where Slurm is the standard job scheduler. The distributed-downloader was used to construct portions of the TreeOfLife dataset, which aggregates species images from across GBIF's network of over 500 contributing institutional servers. Unlike img2dataset, the distributed-downloader operates in an environment where the target servers are heterogeneous in capacity, making the presence of per-host rate control a necessity.

However, the distributed-downloader shares several limitations with img2dataset. It does not implement per-host congestion control or adaptive rate pacing; downloads proceed at the maximum rate the worker can sustain regardless of server load signals. Its reliance on MPI requires a fixed world size determined at job submission time; workers cannot join or leave the workflow dynamically, and the compute resources must belong to a single Slurm-managed cluster. This excludes scenarios in which a research group might combine university HPC allocations, NSF cloud instances such as Jetstream2, and local workstations into a single coordinated download workflow.

The limitations of both systems converge on two gaps. First, neither tool implements adaptive per-host request rate control informed by server response signals such as TTFB latency, HTTP 429 and 503 codes, or connection-level errors. Second, neither supports distributed execution across heterogeneous compute environments without requiring a shared filesystem, a common job scheduler, or synchronized worker start times. The practical consequence of these gaps is that academic research groups attempting to construct training datasets from server-diverse sources, the typical case for biodiversity, ecology, and regional medical imaging, face systematic download failures concentrated on the smallest institutional servers. Because these servers disproportionately host images from rare or geographically restricted taxa, the resulting dataset underrepresents precisely the classes that are most ecologically or clinically relevant, possibly introducing a bias into the training data before any model is trained.

2.2 Cost-Sensitive Deep Learning

Although cost-sensitive classification and class-imbalanced classification are frequently treated as interchangeable in the deep learning literature, they are mathematically distinct problems with different objectives and different solutions. Class imbalance refers to the setting in which training class frequencies are unequal; the standard remedies, oversampling, undersampling, and frequency-derived loss reweighting, aim to equalize each class's influence on the learned decision boundary. Cost-sensitive classification, by contrast, addresses the setting in which different misclassification errors carry different real-world penalties. These penalties can be encoded in an $N \times N$ cost matrix $\mathbf{C}$ whose entry $C[k, j]$ specifies the cost incurred when a sample of true class j is predicted as class k [16]. These costs are determined by domain knowledge (clinical severity, ecological consequence, economic impact) and need not bear any relationship to class frequencies. The two problems can co-occur, but addressing one does not address the other: a class-balanced dataset still requires cost-sensitive methods if certain misclassification pairs are more consequential than others [16, 23].

The theoretical foundations for cost-sensitive classification are well established. For binary classification, Elkan [16] proved that the Bayes-optimal decision threshold under an arbitrary 2×2 cost matrix

is

$$p^* = \frac{C_{\text{FP}} - C_{\text{TN}}}{C_{\text{FP}} - C_{\text{TN}} + C_{\text{FN}} - C_{\text{TP}}}, \tag{1}$$

where C_{FP}, C_{TN}, C_{FN}, and C_{TP} denote the costs of false positive, true negative, false negative, and true positive decisions, respectively. This result shows that any economically coherent binary cost matrix reduces to a single degree of freedom controlling the decision threshold. For the multiclass setting with N classes, Bayes decision theory prescribes the cost-minimizing prediction rule

$$\hat{y}(\mathbf{x}) = \underset{k \in \{1,\ldots,N\}}{\arg\min} \sum_{j=1}^{N} C[k, j] \cdot P(Y = j \mid \mathbf{x}), \tag{2}$$

which selects the class that minimizes expected misclassification cost given the posterior class probabilities. This rule requires both a well-calibrated probabilistic classifier and an explicit cost matrix, and is the starting point for all cost-sensitive methods [16, 24]. Domingos [24] operationalized this principle in MetaCost, a wrapper method that uses bagging to estimate class probabilities, then relabels each training example with the class minimizing expected cost under an arbitrary $N \times N$ matrix. MetaCost handles the full multiclass cost-sensitive setting, yet no direct deep learning adaptation exists.

Dmochowski et al. [25] provided the key theoretical bridge to loss-function-based approaches by proving that the cost-weighted negative log-likelihood

$$\mathcal{L}_{\text{CW}} = -\frac{1}{n} \sum_{i=1}^{n} w_{y_i} \log p_{y_i}(\mathbf{x}_i) \tag{3}$$

is a tight, convex upper bound on the empirical misclassification cost, where w_{y_i} is a per-class weight derived from the cost matrix. This result justifies cost-weighted cross-entropy as a surrogate loss for cost-sensitive training. However, Dmochowski et al. also showed that under model misspecification, the risk-minimizing solution varies with the cost ratio, meaning that a single set of learned weights may not be optimal across all operating points.

Despite these mature theoretical foundations, the translation to deep learning has been limited. Chung, Lin, and Yang [21] published the first work addressing cost-sensitive deep learning with explicit $N \times N$ cost matrices. Their CSDNN framework introduced the SOSR (Softmax-based Output and Sensitivity-aware Regularization) loss, which embeds cost information directly into training, and combined it with cost-aware autoencoders for pre-training. Evaluated on MNIST, CIFAR-10, USPS, and SVHN with randomized proportional cost matrices, CSDNN demonstrated that incorporating cost information during training outperforms approaches that apply cost-sensitive decisions only at prediction time. The cost matrices used were arbitrary and not derived from class frequencies, making this work genuinely cost-sensitive rather than a class-imbalance rebalancing method.

Galdran et al. [15] proposed a cost-sensitive regularization approach applied to diabetic retinopathy grading. Rather than replacing the standard training loss, they introduced an additive cost-sensitive regularizer

$$\mathcal{L}_{\text{CS}}(\mathbf{x}, y) = \sum_{k=1}^{N} M[y, k] \cdot p_k(\mathbf{x}), \tag{4}$$

where M is an $N \times N$ cost matrix and $p_k(\mathbf{x})$ is the predicted probability for class k. This term penalizes the model for assigning probability mass to classes whose confusion with the true class y is costly. The total

training loss combines a standard objective with the regularizer:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{base}} + \lambda \cdot \mathcal{L}_{\text{CS}}, \tag{5}$$

where λ controls the regularization strength. A key empirical finding was that using the cost-sensitive term as the sole training loss causes CNNs to collapse to trivial solutions, predicting a single class for all inputs. This training instability necessitates the regularization formulation. Applied with distance-based cost matrices ($M[i, j] = |i - j|$ for L1 and $M[i, j] = (i - j)^2$ for L2) reflecting disease severity, the approach achieved a 3–5% improvement in quadratic-weighted kappa on the EyePACS diabetic retinopathy dataset [15]. This work remains one of very few open-source PyTorch implementations supporting arbitrary $N \times N$ cost matrices.

A result with significant implications for this research was established by Chen et al. [26], who identified a fundamental obstacle to cost-sensitive training in modern deep networks. They proved that when overparameterized neural networks interpolate training data (achieving zero training loss), cost-weighted empirical risk minimization collapses to standard ERM, because the loss is zero regardless of the cost weights applied. Formally, if $\mathcal{L}_{\text{CW}}(\theta^*) = 0$ for parameters θ^* that interpolate the data, then the gradient $\nabla_\theta \mathcal{L}_{\text{CW}}(\theta^*) = \mathbf{0}$ irrespective of the values of the cost weights w_y. This result undermines cost-weighted cross-entropy, per-class loss reweighting, and any other approach that modifies the loss by multiplicative cost factors when applied to models with sufficient capacity to memorize the training set. To address this, Chen et al. proposed CSADA (Cost-Sensitive Adversarial Data Augmentation), a bi-level optimization framework in which the inner loop generates adversarial examples targeted at high-cost misclassification pairs via multi-step gradient ascent, and the outer loop trains on both the original and adversarial examples with cost-aware penalties. Evaluated on MNIST, CIFAR-10, and a pharmacy medication image dataset with expert-assigned pairwise costs, CSADA maintained overall accuracy while reducing average test cost by 27–48% relative to standard training [26].

Several specific gaps define the research agenda for cost-sensitive deep learning with modern architectures. No Vision Transformer, ConvNeXt, or EfficientNet architecture has been paired with explicit $N \times N$ cost matrices, and no SSL (self-supervised learning) foundation model backbone, such as DINOv3, CLIP, or their successors, has been fine-tuned with a cost-sensitive objective. No published work combines cost-sensitive regularization with logit adjustment, and no work has attempted to extend logit adjustment from class priors to arbitrary cost matrices. Finally, the overparameterization result of Chen et al. suggests that parameter-efficient fine-tuning methods, which do not interpolate the training data, may be essential for cost matrices to meaningfully influence the learned decision boundary, but this interaction has not been investigated. The practical consequence is that a domain expert, a clinician who knows that undergrading proliferative diabetic retinopathy is far more dangerous than overgrading mild disease, or an ecologist who knows that confusing an invasive species with a native species has different management consequences than the reverse, currently has no existing tool to encode that knowledge into a modern deep learning training pipeline.

3. Proposed Study

3.1 Overview of Proposed Approach

The proposed research consists of two systems that address complementary stages of the applied image classification pipeline. The first system, FLOW-DC (Flexible Large-scale Orchestrated Workflow for Data Collection) [13], is a distributed image downloading framework that constructs ML-ready datasets from URL manifests hosted across heterogeneous web servers. The second system, NICME model (Non-Identical

Matrix Embedding), is a cost-sensitive image classification framework that trains deep learning models using explicit $N \times N$ misclassification cost matrices on modern vision backbones. The two systems are connected by a shared workflow: FLOW-DC produces the training datasets on which NICME model operates, and the class-distribution fidelity of the downloaded dataset directly affects the validity of the cost-sensitive training that follows.

FLOW-DC extends the manager-worker paradigm of TaskVine [27] with an application-layer congestion control algorithm called PAARC (Policy-Aware Adaptive Request Controller) [13]. The manager machine partitions a dataset manifest into fixed-length chunks using a host-dispersed partitioning algorithm that separates URLs from the same host across distinct chunks, then assigns these chunks to worker machines as TaskVine tasks.

NICME model is a PyTorch-based image classification framework that extends the standard training pipeline with support for user-defined cost matrices. The proposed research will extend this framework in two directions. First, we will integrate the foundation model backbone DINOv3 [8] with ViT and ConvNeXt using parameter-efficient fine-tuning via LoRA [28]. Second, we will develop and evaluate the combined loss function that integrates a cost-sensitive regularizer [15] and logit adjustment [19], adapting the logit adjustment term to operate with cost-matrix-derived offsets rather than class priors.

3.2 FLOW-DC: Distributed Downloading with PAARC

FLOW-DC addresses Objective 1 through three interacting components: a host-aware dataset partitioning algorithm, the PAARC congestion control algorithm operating on each worker, and the TaskVine-based distributed orchestration layer. This section details each component. The full system design and implementation are described in Deng et al. [13].

Host-Aware Partitioning. Before distributing work to workers, the manager partitions the dataset manifest into chunks that disperse URLs from the same host server across distinct partitions. This dispersion is important because it prevents any single worker from concentrating requests on one host, distributing the load across the worker pool and reducing the likelihood of server-side throttling. Let $H = \{h_1, h_2, \ldots, h_m\}$ denote the set of m unique hosts extracted from the manifest, where each host h_i has URL count c_i and URL set $U_i = \{u_{i,1}, u_{i,2}, \ldots, u_{i,c_i}\}$. Let k denote the desired number of partitions (a hyperparameter). For each host, the algorithm assigns its URLs to partitions via offset round-robin allocation, where the starting partition rotates per host to balance the distribution of remainder URLs:

$$\forall\, u_{i,j} \in U_i : \quad u_{i,j} \longrightarrow P_{((j-1+\delta_i) \bmod k)+1}, \quad \delta_i = (i-1) \bmod k. \tag{6}$$

The per-host offset δ_i rotates the round-robin starting partition for each successive host, ensuring that the remainder URLs (those arising when c_i is not evenly divisible by k) are spread across different partitions rather than accumulating in the same low-indexed partitions. This provides two simultaneous guarantees: each partition receives exactly $\lfloor c_i/k \rfloor$ or $\lceil c_i/k \rceil$ URLs from any single host h_i, providing optimal per-host dispersion, and global partition sizes remain balanced to within at most $\lfloor m/k \rfloor$ URLs of each other in the worst case. When $c_i \geq k$, every partition receives at least one URL from host h_i, ensuring no single worker bears a disproportionate share of requests to any server.

PAARC Algorithm. The Policy-Aware Adaptive Request Controller operates on each worker node and dynamically adjusts the number of concurrent HTTP requests issued to each host server. The primary control variable is the concurrency C, the number of simultaneous in-flight requests. The download rate R emerges

naturally from Little's Law, $R = C/T$, where T is the mean response time [13]. This concurrency-primary design is self-regulating: when server latency increases, throughput decreases automatically without requiring an explicit rate adjustment.

PAARC monitors time-to-first-byte (TTFB) as its primary latency signal. For each control interval, the controller computes the 10th, 50th, and 95th percentiles of successful request TTFBs and smooths them with an exponential moving average:

$$p_k^{(t)} = \alpha \cdot p_k^{\text{raw}} + (1 - \alpha) \cdot p_k^{(t-1)}, \tag{7}$$

where p_k denotes the k-th percentile and $\alpha = 0.3$ is a hyperparameter controlling the smoothing factor. The baseline latency RTprop is defined as the minimum observed 10th-percentile TTFB over a sliding window:

$$\text{RTprop} = \min_{t' \in [t-W,\, t]} p_{10}^{(t')}, \tag{8}$$

where $W = 10$ seconds is a hyperparameter. Latency degradation is detected when median or tail latencies exceed threshold multiples of RTprop:

$$\text{degraded} = \big(p_{50} > \text{RTprop} \times \theta_{50}\big) \;\vee\; \big(p_{95} > \text{RTprop} \times \theta_{95}\big), \tag{9}$$

with hyperparameters $\theta_{50} = 1.5$ and $\theta_{95} = 2.0$.

PAARC State Machine. PAARC implements a five-state finite state machine governing concurrency adjustment decisions [13]. In the **INIT** state, the controller operates at the hyperparameter C_{init} concurrent requests and collects initial TTFB samples to establish the RTprop baseline. Upon collecting sufficient samples, the controller transitions to **STARTUP**, where it discovers the maximum sustainable concurrency through additive growth. Each control interval, the concurrency is increased by a fixed hyperparameter increment $\Delta C_{\text{startup}}$. STARTUP terminates when either an overload signal occurs (transitioning to BACKOFF) or a latency-based plateau is detected; that is, when the latency degradation condition in Equation 9 is triggered using elevated hyperparameter thresholds $\theta_{50}^{\text{startup}}$ and $\theta_{95}^{\text{startup}}$. Upon plateau detection, PAARC sets the concurrency ceiling C_{ceiling} to the current concurrency and computes the operating point:

$$C_{\text{operating}} = \lfloor C_{\text{ceiling}} \times \mu \rfloor, \tag{10}$$

where $\mu = 0.75$ is the utilization factor (a hyperparameter), then transitions to the steady-state **PROBE_BW** phase. In PROBE_BW, the controller maintains concurrency at or below $C_{\text{operating}}$ and periodically probes for additional capacity via conservative additive increase. If an overload signal occurs, the ceiling is revised downward by a multiplicative decrease hyperparameter $\beta = 0.5$ and the controller transitions to **BACKOFF**. The controller periodically enters **PROBE_RTT** to refresh the RTprop estimate by reducing concurrency by a fraction ρ of the current value:

$$C_{\text{probe}} = \max\big(C_{\text{min}},\ \lfloor C \times \rho \rfloor\big), \tag{11}$$

where $\rho = 0.5$ and $C_{\text{min}} = 1$ are hyperparameters, collecting fresh latency samples at reduced load, then gradually restoring concurrency over multiple steps to avoid burst-induced congestion. BACKOFF handles recovery from overload conditions (HTTP 429, 5xx, or connection-level errors) by enforcing a cooldown period before restoring concurrency to $C_{\text{operating}}$ and returning to PROBE_BW. This design draws on BBR's

model-based approach to congestion control [29] but adapts its mechanisms for the application layer, where congestion signals are HTTP status codes and TTFB measurements rather than packet-level acknowledgements.

Distributed Execution. The manager instantiates a TaskVine workflow in which each partition is submitted as an independent task with declared input files (partition parquet, download scripts, configuration JSON) and output files (e.g., compressed tar archive, JSON summary). Workers connect to the manager utilizing the manager's IP and receive task assignments from the pending queue. TaskVine handles input file transfer, task scheduling, and result collection. Workers that disconnect mid-workflow do not cause data loss; TaskVine detects the disconnection and re-queues incomplete tasks for available workers. This elastic participation model requires only network reachability between the manager and workers: no shared filesystem, no common job scheduler, and no synchronized start times [27].

3.3 NICME: Cost-Sensitive Learning using Logit Adjustment and Regularization

The core algorithmic contribution of NICME model is a loss function that integrates two cost-sensitive mechanisms, logit adjustment and cost-sensitive regularization, into a single training objective. Each mechanism addresses a different aspect of cost-aware learning.

The first component is a cost-matrix-driven logit adjustment. For each training sample, the model's predicted class $\hat{k} = \arg\max_k f_k(\mathbf{x})$ is compared to the true class y. For misclassified samples ($\hat{k} \neq y$), the logit corresponding to the predicted class is increased proportionally to both the cost of that specific misclassification and the logit gap between the predicted and true classes:

$$\tilde{f}_{\hat{k}}(\mathbf{x}) = f_{\hat{k}}(\mathbf{x}) + C[y, \hat{k}] \cdot \left| f_{\hat{k}}(\mathbf{x}) - f_y(\mathbf{x}) \right|, \tag{12}$$

where $C[y, \hat{k}]$ is the entry of the user-defined cost matrix corresponding to misclassifying true class y as class $\hat{k}$, and $f_k(\mathbf{x})$ denotes the logit for class k. For correctly classified samples, the logits are unchanged. This adjustment amplifies the cross-entropy gradient for high-cost misclassifications: when $C[y, \hat{k}]$ is large, the modified logit $\tilde{f}_{\hat{k}}$ makes the model appear more confident in the costly wrong prediction, producing a stronger corrective gradient during backpropagation. The cross-entropy loss is then computed on the adjusted logits:

$$\mathcal{L}_{\text{LA}} = -\frac{1}{n} \sum_{i=1}^{n} \log \frac{\exp(\tilde{f}_{y_i}(\mathbf{x}_i))}{\sum_{k=1}^{N} \exp(\tilde{f}_k(\mathbf{x}_i))}. \tag{13}$$

The second component is a cost-sensitive regularizer introduced by Galdran et al. [15], which penalizes the model for assigning probability mass to classes whose confusion with the true class is costly. This term operates on the original (non-adjusted) logits to ensure the two gradient paths are independent:

$$\mathcal{L}_{\text{CS}} = \frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{N} \bar{M}[y_i, k] \cdot \sigma_k(\mathbf{x}_i), \tag{14}$$

where $\bar{M}$ is the cost matrix normalized to $[0, 1]$ and $\sigma_k(\mathbf{x}_i) = \text{softmax}(f(\mathbf{x}_i))_k$ is the predicted probability for class k computed from the original logits. The normalization prevents unbounded penalty magnitudes when cost matrix entries span several orders of magnitude.

The combined loss function is:

$$\mathcal{L}_{\text{NICME}} = \mathcal{L}_{\text{LA}} + \lambda_{\text{eff}} \cdot \mathcal{L}_{\text{CS}}, \tag{15}$$

where λ_{eff} is the effective regularization weight (a hyperparameter). To prevent the regularization term from destabilizing early training, a failure mode identified by Galdran et al. [15], who found that cost-sensitive losses alone cause CNNs to collapse to trivial solutions, we apply a linear warmup schedule:

$$\lambda_{\text{eff}}(e) = \lambda \cdot \min\left(1,\ \frac{e}{E_{\text{warmup}}}\right), \tag{16}$$

where e is the current epoch and the hyperparameters E_{warmup} and λ specify the warmup duration in epochs and the target regularization strength, respectively. During warmup, the model trains primarily on $\mathcal{L}_{\text{LA}}$; the regularizer's contribution increases linearly until reaching full strength at epoch E_{warmup}.

Backbone Architecture and Parameter-Efficient Fine-Tuning. NICME model currently implements ResNet-50 [2] and ConvNeXt [5] backbones using custom base classes that decouple the model architecture from HuggingFace's transformers library. Both architectures load ImageNet-pretrained weights and replace the final classification head with a linear layer matching the number of target classes. Stage freezing is supported: setting the hyperparameter s (number of frozen stages) freezes all parameters in the first s convolutional stages, restricting gradient updates to the later stages and the classification head.

The proposed extension integrates DINOv3 [8] pretraining combined with ConvNeXt and ViT as backbone architectures via parameter-efficient fine-tuning using LoRA (Low-Rank Adaptation) [28]. LoRA constrains the weight update to a low-rank decomposition: for a pretrained weight matrix $\mathbf{W}_0 \in \mathbb{R}^{d\times d}$, the adapted weight is

$$\mathbf{W} = \mathbf{W}_0 + \mathbf{BA}, \tag{17}$$

where $\mathbf{B} \in \mathbb{R}^{d\times r}$ and $\mathbf{A} \in \mathbb{R}^{r\times d}$ with rank $r \ll d$. Only $\mathbf{A}$ and $\mathbf{B}$ are trainable; $\mathbf{W}_0$ remains frozen. This reduces the number of trainable parameters to 0.1–2% of the full model, which provides two advantages for cost-sensitive learning. First, it avoids the overparameterization collapse identified by Chen et al. [26]: because the vast majority of parameters are frozen, the model cannot fully interpolate the training data, and the cost weights in $\mathcal{L}_{\text{NICME}}$ retain their influence on the learned decision boundary. Second, the LIFT result [30] demonstrated that lightweight fine-tuning with logit-adjusted loss preserves the class-conditional feature distributions learned during pretraining, which heavy fine-tuning destroys. LoRA applied to the linear projection layers of DINOv3 and ConvNeXt, with rank $r \in \{4, 8, 16\}$ as a hyperparameter, is the proposed default configuration.

Post-hoc temperature scaling will be applied as a final calibration step. Given a trained model with logits $f(\mathbf{x})$, the calibrated probabilities are $\sigma(f(\mathbf{x})/T)$ where the temperature $T > 0$ is optimized on a held-out validation set by minimizing the negative log-likelihood [31].

The primary metric used to evaluate whether the trained model satisfies the user-specified cost preferences is the average test cost (ATC), defined as the mean per-sample cost incurred by the model's predictions on the test set:

$$\text{ATC} = \frac{1}{n}\sum_{i=1}^{n} C[\hat{y}_i, y_i], \tag{18}$$

where n is the number of test samples, $\hat{y}_i$ is the model's prediction for sample i, y_i is the true class, and $C[\hat{y}_i, y_i]$ is the corresponding entry of the user-specified cost matrix. ATC is the quantity that hyperparameter optimization and final model selection will minimize. Alongside ATC, we will additionally track per-class recall stratified by cost category (high, medium, low), and overall classification accuracy, to confirm that cost reductions are not achieved by degrading general classification performance or calibration.

3.4 Prior Work and Preliminary Implementations

The research proposed in this document is inspired by our work AraNet, a two-stage convolutional neural network for spider species classification [14]. The lessons from AraNet directly motivated both FLOW-DC and NICME model.

AraNet: Spider Species Classification via Multi-Phase Deep Learning. AraNet was developed to address the problem of automated spider species identification, a task with applications in habitat monitoring and emergency medical treatment [14]. Two practical challenges encountered during the AraNet project directly motivated the present research.

The first was dataset construction. Assembling the iNatSpiders1k training set required downloading 548,689 images from iNaturalist's AWS-hosted repository. While iNaturalist serves images from a single high-capacity S3 endpoint, expanding the dataset to utilize other biodiversity data sources, particularly GBIF, which aggregates records across over 500 institutional servers with varying capacity and rate-limiting policies, revealed the limitations of existing download tools. This experience motivated the development of FLOW-DC and its PAARC congestion control algorithm.

The second challenge was the recognition that standard accuracy-maximizing training does not capture the asymmetric consequences of misclassification in safety-relevant domains. In the Australian spider classification task, misidentifying a venomous species such as the Sydney funnel-web (*Atrax robustus*) as a non-dangerous species could delay life-saving medical intervention, while the reverse error (classifying a harmless species as dangerous) results in unnecessary caution but no physical harm. This observation motivated the development of NICME's cost-matrix-based training framework and directly informs the design of the combined loss function described in Section 3.3.

FLOW-DC: Production Deployment for Biotrove Dataset FLOW-DC has been deployed at production scale for the construction of the Biotrove dataset, a large-scale biodiversity image collection contributed to the NeurIPS 2024 Datasets and Benchmarks Track [12]. In this deployment, FLOW-DC downloaded 38.7 million images totaling 13 TiB from GBIF's network of institutional servers using 10 Jetstream2 cloud workers. The core architecture of FLOW-DC including the PAARC algorithm has been constructed [13].

NICME Preliminary Model The prototype NICME model framework has been implemented in PyTorch using the HuggingFace Trainer API. The framework currently supports ResNet-50 [2] and ConvNeXt [5] backbones with configurable stage freezing, and utilizes an augmented cross-entropy loss that integrates logit adjustment with cost matrix values. The cost matrix is accepted as a user-defined input and propagated throughout model training.

4. Proposed Experiments

The experimental plan is organized around the two research objectives stated in Section 1.2. For Objective 1 (FLOW-DC), the evaluation focuses on rate-limit compliance, server-fault resilience, and scaling behavior rather than raw download throughput, since both FLOW-DC and its primary baseline are ultimately bottlenecked by available network bandwidth. For Objective 2 (NICME model), the evaluation focuses on whether the proposed cost-sensitive loss function lowers the average test cost (ATC) induced by a user-specified cost matrix, while maintaining overall classification accuracy. Each objective is evaluated against a set of baselines selected to isolate the contribution of specific design decisions.

4.1 Baseline Comparisons

FLOW-DC Baselines. The primary baseline to evaluate FLOW-DC is **img2dataset** [22], the most widely deployed open-source tool for large-scale image downloading. As detailed in Section 3.2, img2dataset uses synchronous thread-based HTTP via `urllib.request` with no per-host congestion control, treats HTTP 429 responses as permanent failures, and requires Apache PySpark for distributed execution. We believe img2dataset is the appropriate primary baseline because it represents the current standard against which any new downloading tool must be measured, and its published benchmarks (approximately 90 samples per second per CPU core, scaling to roughly 10,000 images per second across a 10-node Spark cluster [10]) provide the throughput reference for establishing parity.

NICME Model Baselines. Three baselines are used to evaluate NICME model's cost-sensitive loss function, each isolating a different aspect of the proposed approach. All baselines will be evaluated using the same backbone architecture, dataset, and training hyperparameters to ensure that differences in performance are attributable to the loss function rather than to confounding factors.

The first baseline is **standard cross-entropy** (CE), which assigns equal penalty to all misclassification errors:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{n}\sum_{i=1}^{n} \log \frac{\exp(f_{y_i}(\mathbf{x}_i))}{\sum_{k=1}^{N} \exp(f_k(\mathbf{x}_i))}. \tag{19}$$

This is the cost-insensitive baseline against which any cost-sensitive method must demonstrate improvement in average test cost (ATC) as defined in Equation 18. Standard CE is expected to minimize overall error rate but not to preferentially avoid high-cost misclassification pairs.

The second baseline is **cost-sensitive regularization alone** (NULS-CS), as proposed by Galdran et al. [15]. This uses the loss $\mathcal{L}_{\text{base}} + \lambda \cdot \mathcal{L}_{\text{CS}}$ with the regularizer defined in Equation 4 but without the logit adjustment component. NULS-CS achieved a 3–5% improvement in quadratic-weighted kappa on diabetic retinopathy grading using a ResNeXt-50 backbone [15]. This baseline isolates the contribution of the logit adjustment term: if NICME model outperforms NULS-CS, the improvement is attributable to the cost-matrix-driven logit adjustment described in Section 3.3.

The third baseline is **logit-adjusted cross-entropy** as proposed by Menon et al. [19], which adds the log class prior $\log \pi_y$ to each logit. Although this method was designed for long-tailed class distributions rather than cost-sensitive classification, it is the closest published method to NICME model's logit adjustment mechanism and serves as a reference for the contribution of replacing class-prior offsets with cost-matrix-derived adjustments. On balanced datasets where $\pi_y = 1/N$ for all classes, this baseline reduces to standard CE, providing an additional control.

4.2 Experimental Methodology

FLOW-DC Experiments. FLOW-DC will be evaluated using two differing manifests that separate content delivery network (CDN) based workloads from server-diverse workloads. The first manifest is a CDN-dominated control drawn from DataComp small [32], which contains 12.8 million image-text pairs derived from Common Crawl. Because Common Crawl-sourced URLs terminate predominantly on high-capacity CDN infrastructure, this manifest tests throughput parity between FLOW-DC and img2dataset under conditions where congestion control provides low benefit; neither tool should trigger rate limiting. The second manifest is a server-diverse experimental condition constructed by filtering the GBIF occurrence snapshot to a subset of approximately 1–3 million URLs spanning 50–100 distinct institutional servers. This manifest tests PAARC's core value proposition: adaptive throttling on capacity-limited servers.

Both FLOW-DC (with PAARC enabled), and img2dataset will be run on both manifests. For single-machine experiments, the primary platform is the University of Arizona HPC Puma cluster. For distributed experiments, 10 Jetstream2 cloud instances will be used (matching the Biotrove dataset generation configuration [13]). Scaling efficiency will be measured at $N = 1, 2, 5, 10$ workers.

FLOW-DC Metrics. The primary throughput metric is the aggregate download rate $T_{\text{agg}} = N_{\text{success}}/t_{\text{download}}$ (images per second), where t_{download} excludes partitioning, packaging, and transfer time. Both images-per-second and TiB-per-hour will be reported, with TiB-per-hour preferred for cross-study comparison because image sizes differ across datasets. The primary metric for evaluating rate-limit compliance and server-fault resilience is the per-host server-side failure rate (SFR), which aggregates all responses indicative of server throttling or overload: HTTP 429 (Too Many Requests), 5xx server errors (500, 502, 503, 504), connection resets, and request timeouts:

$$\text{SFR}(h) = \frac{N_{429}(h) + N_{5xx}(h) + N_{\text{timeout}}(h) + N_{\text{reset}}(h)}{N_{\text{total}}(h)} \times 100\%. \tag{20}$$

Scaling efficiency is defined as:

$$E(N) = \frac{T_{\text{agg}}(N)}{N \cdot T_{\text{agg}}(1)}, \tag{21}$$

where $T_{\text{agg}}(N)$ is the aggregate throughput with N workers. Output dataset quality is measured by the class distribution fidelity (CDF):

$$\text{CDF} = 1 - D_{\text{KL}}\left(P_{\text{downloaded}} \parallel P_{\text{requested}}\right), \tag{22}$$

where $P_{\text{downloaded}}$ and $P_{\text{requested}}$ are the class frequency distributions of successfully downloaded images versus the original manifest.

NICME Model Experiments. NICME model will be evaluated on four datasets of increasing complexity, each selected to provide a domain-motivated cost matrix. The first is a binary spider classification dataset (2 classes, 500 images per class) distinguishing black widows (*Latrodectus*) from false widows (*Steatoda*), two genera that are frequently confused due to similar morphology but carry vastly different medical consequences. The 2×2 cost matrix assigns high cost to misclassifying a black widow as a false widow (risking delayed antivenom treatment) and low cost to the reverse conservative error. This dataset serves as the simplest baseline for validating the cost-sensitive loss function in a controlled binary setting. The second is the AusSpiders dataset (9 classes, 64,291 images) [14], which extends the binary venomous versus non-venomous cost structure to a multiclass setting. The class distribution will be balanced by downsampling to equalize class frequencies, isolating the effect of the cost matrix from class-imbalance confounds. The third is the EyePACS diabetic retinopathy grading dataset [15], which provides a 5-class ordinal structure (grades 0–4) with clinically motivated asymmetric costs: undergrading severe disease (e.g., predicting grade 0 for a grade 4 sample) carries higher cost than overgrading mild disease. This dataset enables direct comparison with the NULS-CS baseline of Galdran et al. [15], who reported QWK scores on the same data. The fourth is the Pharmacy Medication Image (PMI) dataset (20 classes, 13,955 images) [26], which contains top-down pill images identified by National Drug Code (NDC). Expert costs were assigned by a pharmacist to four critical medication-confusion pairs based on clinical danger; for example, confusing Amiodarone (a cardiac drug) with Allopurinol (a gout medication) risks pulmonary toxicity. Non-critical pairs are assigned unit cost. This dataset provides the most complex cost structure and enables direct comparison with the CSADA baseline of

Chen et al. [26].

For each dataset, all baselines and the proposed $\mathcal{L}_{\text{NICME}}$ loss will be trained using the same backbone, the same data splits (80/10/10 train/validation/test, stratified by class), and the same training hyperparameters (AdamW optimizer, cosine annealing, early stopping on validation loss). To ensure statistical reliability, each configuration will be run across a minimum of 5 random seeds.

Optimal hyperparameters will be determined via Bayesian hyperparameter optimization (HPO) using Optuna [33] with the Tree-structured Parzen Estimator (TPE) sampler. For each dataset and backbone combination, Optuna will search over the cost-sensitive hyperparameters introduced in Section 3.3 (the regularization strength λ, the warmup duration E_{warmup}, and the LoRA rank r) as well as standard training hyperparameters including learning rate, weight decay, and batch size. The optimization objective is to minimize ATC on the validation split, ensuring that the selected hyperparameters directly optimize the cost-sensitive goal rather than overall accuracy.

NICME Model Metrics. The primary metric is the average test cost (ATC) as defined in Equation 18. The cost reduction ratio (CRR) will be reported as:

$$\text{CRR} = \frac{\text{ATC}_{\text{baseline}} - \text{ATC}_{\text{proposed}}}{\text{ATC}_{\text{baseline}}}. \tag{23}$$

Per-class recall stratified by cost category (high, medium, low) will be reported as a secondary metric. Overall classification accuracy and F1-score will also be tracked to ensure that cost-sensitive training does not trivially improve high-cost recall by degrading performance on other classes. All results will be reported as mean ± standard deviation across seeds, with 95% confidence intervals.

Hardware. FLOW-DC single-machine experiments will use the UofA HPC Puma cluster (AMD EPYC 7642, 25 Gb/s Ethernet); distributed experiments will use up to 10 Jetstream2 cloud instances (up to 128 cores, 100 Gbps per node). NICME model training will be conducted on the University of Arizona HPC GPU resources, primarily Nvidia V100S GPUs (32 GB) and Nvidia A100 GPUs. Local experiments will utilize an Nvidia RTX 5090 GPU (32 GB) for rapid prototyping.

5. Risks and Alternatives

Overparameterization collapse under LoRA. The use of LoRA to preserve cost-weight influence has not been empirically verified for cost-sensitive objectives. If LoRA-adapted models still achieve sufficiently low training loss that cost weights become ineffective, we will fall back to investigating cost-sensitive post-hoc decision rules using the Bayes-optimal rule (Equation 2), a two-stage approach that requires no modification to the training loss [16].

Combined loss does not outperform individual components. The logit adjustment and regularization terms in $\mathcal{L}_{\text{NICME}}$ may interfere destructively rather than complement each other. If no configuration of λ and E_{warmup} yields improvement over the individual components, the contribution shifts to optimization and augmentation on the single best performing method between regularization and logit adjustment.

6. Resources

All computational, storage, and software resources for the project are accessible through existing institutional allocations and personal hardware. Compute time on the University of Arizona HPC center (Puma CPU nodes

and the shared GPU allocation providing Nvidia V100S and A100 GPUs) is available under the advisor's standing allocation, and NSF Jetstream2 cloud instances for distributed FLOW-DC experiments are covered under an existing usage-based allocation.

Software. All software used in this research is open-source and requires no commercial licenses. FLOW-DC is implemented in Python using aiohttp (async HTTP), Polars (data processing), and TaskVine (distributed workflow management) [27]. NICME model is implemented in PyTorch with the HuggingFace Trainer API, Optuna for hyperparameter optimization [33], and the timm library for pretrained vision model access. Version control is managed through GitHub.

Data and Datasets. FLOW-DC experiments will use publicly available URL manifests from GBIF (Parquet format, hosted on AWS Open Data) and DataComp small [32]. NICME model experiments will use a binary black widow versus false widow spider dataset (1,000 images), the AusSpiders dataset (64,291 images, curated from iNaturalist) [14], the EyePACS diabetic retinopathy dataset [15], and the Pharmacy Medication Image (PMI) dataset (13,955 images) [26]. All datasets will be publicly accessible.

7. Proposed Schedule

Table 1: Proposed research timeline.

Month	Task	Deliverable
Spring 2026	Oral Qualification Exam	Qualification Exam Pass
Spring 2026	FLOW-DC final throughput experiments	FLOW-DC results
Spring 2026	FLOW-DC paper draft	FLOW-DC paper
Spring 2026	NICME binary model experiments	NICME preliminary results #1
Summer 2026	Binary Class NICME Paper	NICME paper #1
Summer 2026	NICME multi-class extension experiments	NICME multiclass results
Fall 2026	NICME class imbalance methods integration testing	NICME integration results
Fall 2026	NICME extension paper draft	NICME paper #2
Fall 2026	Dissertation writing	Dissertation paper
Fall 2026	Defense preparation	Dissertation defense

References

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, pp. 1097–1105, 2012.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

[3] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.

[4] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, pp. 10012–10022, 2021.

[5] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11966–11976, 2022.

[6] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 16000–16009, 2022.

[7] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Machine Learning (ICML)*, pp. 8748–8763, 2021.

[8] T. Darcet, M. Oquab, J. Mairal, and P. Bojanowski, "DINOv3: Unifying self-supervised and dense visual learning with gram anchoring," *arXiv preprint arXiv:2508.10104*, 2025.

[9] A. Krizhevsky, "Learning multiple layers of features from tiny images," tech. rep., University of Toronto, 2009.

[10] C. Schuhmann, R. Beaumont, R. Vencu, C. Gordon, R. Wightman, M. Cherti, T. Coombes, A. Katta, C. Mullis, M. Wortsman, P. Schramowski, S. Kundurthy, K. Crowson, L. Schmidt, R. Kaczmarczyk, and J. Jitsev, "LAION-5B: An open large-scale dataset for training next generation image-text models," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022.

[11] A. Torralba and A. A. Efros, "Unbiased look at dataset bias," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 1521–1528, 2011.

[12] C.-H. Yang, B. Feuer, Z. Jubery, Z. K. Deng, A. Nakkab, M. Z. Hasan, S. Chiranjeevi, K. Marshall, N. Baishnab, A. K. Singh, A. Singh, S. Sarkar, N. Merchant, C. Hegde, and B. Ganapathysubramanian, "BioTrove: A large curated image dataset enabling AI for biodiversity," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track (Spotlight)*, 2024.

[13] Z. Deng, N. Merchant, and J. J. Rodriguez, "FLOW-DC: A large-scale distributed downloader utilizing policy-aware adaptive request control," *IEEE Transactions on Big Data*, 2025. Submitted.

[14] Z. Deng and J. J. Rodriguez, "Spider species identification via multi-phase deep learning," *Arthropoda*, vol. 1, pp. 1–17, 2026.

[15] A. Galdran, J. Dolz, H. Chakor, H. Lombaert, and I. B. Ayed, "Cost-sensitive regularization for diabetic retinopathy grading from eye fundus images," in *Medical Image Computing and Computer Assisted Intervention (MICCAI)*, vol. 12370 of *Lecture Notes in Computer Science*, pp. 665–674, Springer, 2020.

[16] C. Elkan, "The foundations of cost-sensitive learning," in *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 973–978, 2001.

[17] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica, "Apache Spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[18] R. Beaumont, "img2dataset: Easily turn large sets of image URLs to an image dataset." https://github.com/rom1504/img2dataset, 2022.

[19] A. K. Menon, S. Jayasumana, A. S. Rawat, H. Jain, A. Veit, and S. Kumar, "Long-tail learning via logit adjustment," in *International Conference on Learning Representations (ICLR)*, 2021.

[20] S. H. Khan, M. Hayat, M. Bennamoun, F. A. Sohel, and R. Togneri, "Cost-sensitive learning of deep feature representations from imbalanced data," *IEEE Trans. Neural Networks and Learning Systems*, vol. 29, no. 8, pp. 3573–3587, 2018.

[21] Y.-A. Chung, H.-T. Lin, and S.-W. Yang, "Cost-aware pre-training for multiclass cost-sensitive deep learning," in *Proceedings of the 25th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 1411–1417, 2016.

[22] R. Beaumont, "img2dataset: Easily turn large sets of image urls to an image dataset." https://github.com/rom1504/img2dataset, 2022. MIT License. Accessed: 2026-03-01.

[23] C. X. Ling and V. S. Sheng, "Cost-sensitive learning," in *Encyclopedia of Machine Learning* (C. Sammut and G. I. Webb, eds.), pp. 231–235, Springer, 2008.

[24] P. Domingos, "MetaCost: A general method for making classifiers cost-sensitive," in *Proceedings of the 5th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 155–164, 1999.

[25] J. P. Dmochowski, P. Sajda, and L. C. Parra, "Maximum likelihood in cost-sensitive learning: Model specification, approximations, and upper bounds," *Journal of Machine Learning Research*, vol. 11, pp. 3313–3332, 2010.

[26] Z. Chen, R. A. Kontar, M. Nouiehed, J. Yang, and B. Lester, "Rethinking cost-sensitive classification in deep learning via adversarial data augmentation," *INFORMS Journal on Data Science*, vol. 4, no. 2, pp. 101–113, 2025.

[27] B. Sly-Delgado, T. S. Phung, C. Thomas, D. Simonetti, A. Hennessee, B. Tovar, and D. Thain, "TaskVine: Managing in-cluster storage for high-throughput data-intensive workflows," in *Workflows in Support of Large-Scale Science (WORKS23), in the Proceedings of SC-W 2023*, pp. 1–11, ACM, 2023.

[28] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

[29] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-based congestion control," *ACM Queue*, vol. 14, no. 5, pp. 20–53, 2017.

[30] S. Tian, L. Qu, W. Wang, S. Niu, and J. Li, "LIFT: Leveraging information from fine-tuning for long-tailed recognition," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[31] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, vol. 70 of *Proceedings of Machine Learning Research*, pp. 1321–1330, PMLR, 2017.

[32] S. Y. Gadre, G. Ilharco, A. Fang, J. Hayase, G. Smyrnis, T. Nguyen, R. Marten, M. Wortsman, D. Ghosh, J. Zhang, E. Orgad, R. Entezari, G. Daras, S. Pratt, V. Ramanujan, Y. Bitton, K. Marathe, S. Mussmann, R. Vencu, M. Cherti, R. Krishna, P. W. Koh, O. Saukh, A. Ratner, S. Song, H. Hajishirzi, A. Farhadi, R. Beaumont, S. Oh, A. Dimakis, J. Jitsev, Y. Carmon, V. Shankar, and L. Schmidt, "DataComp: In search of the next generation of multimodal datasets," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[33] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 2623–2631, 2019.